SUBJECT: Software Design 2	
Code: SDN260S	**Submission Date: 01 November 2026**

Cape Peninsula University of Technology

FACULTY OF ENGINEERING AND THE BUILT ENVIRONMENT

DEPARTMENT OF ELECTRICAL, ELECTRONIC AND COMPUTER ENGINEERING

Bachelor of Engineering Technology in Computer Engineering :2nd year

Check-In 3: Final Integration & Testing

No	Name & Surname	Student Number	Contribution out of 100%
1.	Mnqobi Gumede	240966120	100%
2.	Mixo Makhongele	240342534	100%
3.	Ntsakisi Mahunisi	240926072	100%
4.	Khuthadzo Makuya	240087070	100%
5.	Thingahangwi Tshivhundo	230550878	100%
6.	Mulanga Phalanndwa	222505932	100%
7.	Zwavhudi Ramagadza	250391449	100%
8.	Mukhethwa Bulwana	231234481	100%

Contents

Abstract 4

Introduction 4

1. Problem definition 4

1.1 The real-world problem 4

1.2 Why it matters 4

1.3 UML class diagrams 5

1.4 Classes and methods 7

2. Data flow and relationships 8

3. Core features 9

3.1 Input/output 9

3.2 Processing logic 10

4.3 GUI interface 13

5. Testing and evaluation 13

5.1 Sample files and test cases 13

5.2 Evaluation 14

6. Documentation and demo 14

7. Graduate attribute reflection 24

7.1 GA2 – Application of scientific and engineering knowledge 24

7.2 GA7 – Sustainability and impact of engineering activity 24

9. Conclusion 25

10. References 26

Table of Figures

Figure 1: UML class diagram part 1 core classes. 6
Figure 2: UML class diagram part 2 Repospace style feature classes. 7
Figure 3: Log in screen. 15
Figure 4: Create an account screen. 16
Figure 5: Confirmation that the account SDN2group11 was created. 17
Figure 6: Home screen. 18
Figure 7: Projects screen. 19
Figure 8: Repositories screen before a repository is selected. 20
Figure 9: The first commit saved as v1. 20
Figure 10: Discover screen. 21
Figure 11: A second user sees only the public work of khuthadzomak. 22
Figure 12: My page with statistics badges. 22
Figure 13: Edit profile screen. 23
Figure 14: Settings screen. 24

Abstract

Repospace is a desktop version control system written in Java with a JavaFX interface. A user chooses a folder on their computer and saves numbered snapshots of it, called commits (v1, v2, v3 and so on). For every commit the program shows which files were added, modified or deleted, and any earlier version can be restored into a new folder. Users can also make their work public or private, search for other users and download their public repositories. All data is kept in local SQLite databases, so no database server is needed. The project applies object-oriented design, recursion, collections, file handling, hashing, logging and multithreading.

Introduction

Every software project changes many times before it is finished. Without a proper tool, students keep copies of folders with names like "final" and "final2", or send files to each other, and work is easily lost or overwritten. Git solves this problem, but its commands and ideas are difficult for someone who has never used version control.

Our goal was to build a small, friendly version control tool that shows the main ideas of Git (commit, history, restore and share) through buttons and screens instead of typed commands.

1. Problem definition

1.1 The real-world problem

Students and small teams often manage their project files by hand. They copy whole folders to keep old versions, e-mail files to each other and overwrite each other's changes. When something breaks, there is no record of what changed, who changed it or when, and no easy way to go back to the last version that worked. Repospace solves this by keeping a numbered history of a folder and letting the user see and restore any version.

1.2 Why it matters

Lost or overwritten work usually shows up close to a deadline, when there is no time to redo it. Keeping a full copy of a folder for every version also wastes disk space, because the same unchanged files are stored again. Version control is a basic skill in industry, but beginners struggle with Git's command line. A simple graphical tool lets students learn the core ideas first and then move on to Git with confidence.

1.3 UML class diagrams

The system is shown in two UML class diagrams. The first diagram shows the core classes: the application, the login and home screens, users, profiles, settings, projects, repositories, commits and history, together with the helper classes for storage and security. The second diagram shows the classes we added for the extra GitHub-style features, such as releases, a wiki, discussions, project boards, sponsors and profile statistics.



Figure 1: UML class diagram part 1 core classes.

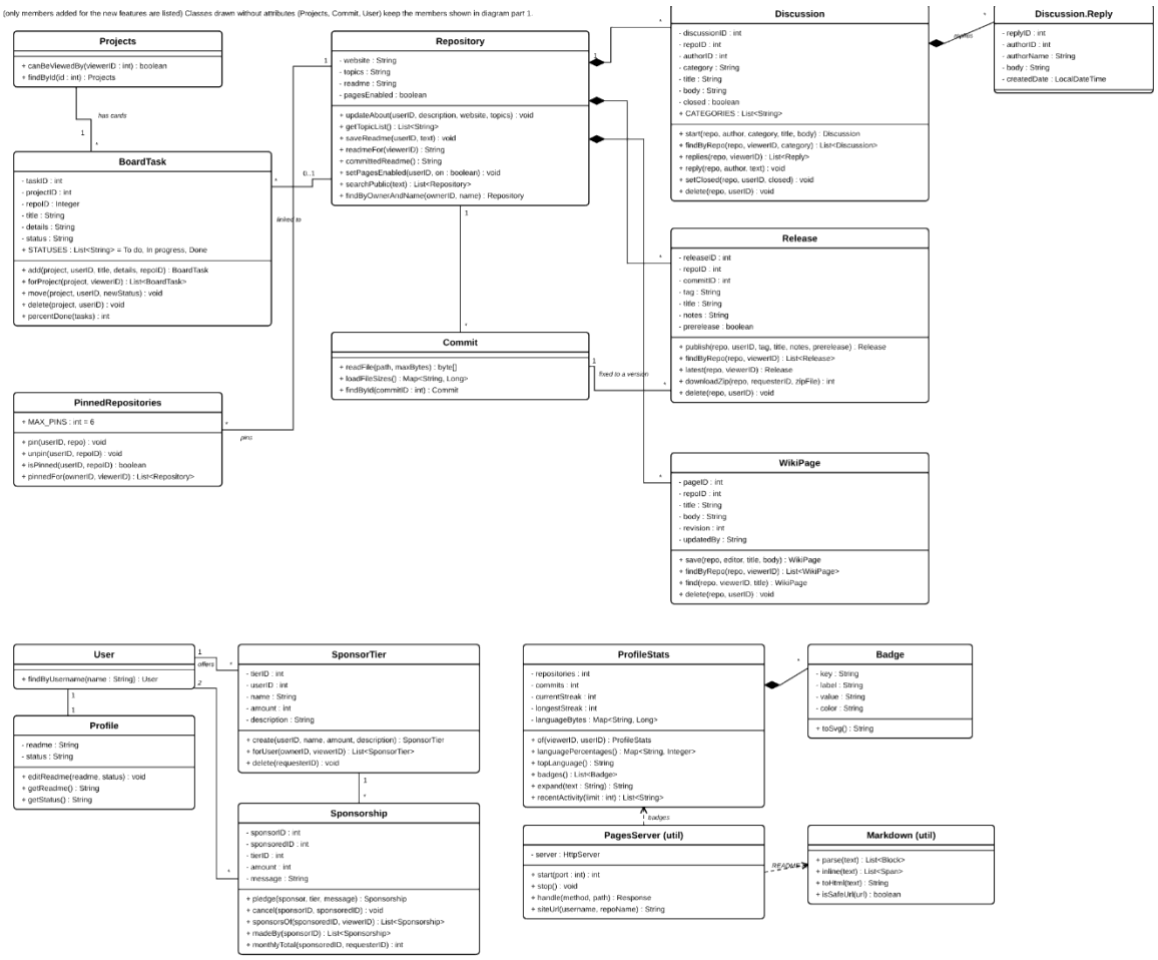


Figure 2: UML class diagram part 2 Repospace style feature classes.

1.4 Classes and methods

Each class has one clear job. The classes are grouped in a Model-View-Controller (MVC) structure.

- **Repospace** is the starting point. It opens the databases, loads the theme and shows the login screen.
- **LoginPageController and RegisterController** check what the user typed, create new accounts and sign users in.
- **HomePageController** shows the sidebar menu and opens the other screens.
- **User** represents an account. It registers new users, checks passwords and searches for other users without ever showing their e-mail address.

- **Profile and Settings** keep the user's bio, picture, visibility, theme, language and notification choices.
- **Projects** keeps a user's projects with a status (Planning, In progress, On hold, Completed) and a visibility.
- **Repository** represents a folder that is being tracked. It can be created, deleted, cloned, made public or private, and downloaded.
- **Commit** is one saved version of a repository. It takes the snapshot, saves it, and can restore or export that version.
- **History** records which files were added, modified or deleted in each commit.
- **SnapshotStore and FileUtil** do the file work: walking through folders, hashing files, comparing versions and restoring files.
- **PasswordUtil** turns passwords into salted hashes so that they are never stored as plain text.
- **DatabaseConnection** opens the four SQLite databases and creates or upgrades their tables.
- **ActivityLogger** writes a log of what happened in the program, including errors and sign-in attempts.

Projects, Profile and Settings also share an interface called **Persistable**, which means each of them can save itself to the database in the same way.

2. Data flow and relationships

When a user saves a new version, the data moves through the system in this order:

1. The user selects a repository, types a message and presses **Commit** on the Repositories screen.
2. The screen's controller starts the work on a background thread, so the window does not freeze.
3. The Commit class asks SnapshotStore to take a snapshot. FileUtil walks through every file in the folder and calculates a SHA-256 hash (a unique fingerprint) of each file's content.

4. Each file is copied into the repository's storage folder under the name of its hash, but only if that content is not stored yet.
5. The new list of files and hashes is compared with the list from the previous version to find the added, modified and deleted files. If nothing changed, the user is told there is nothing to commit.
6. The commit and its history record are saved in one database, and the list of files is saved in another. If the second save fails, the first one is undone so the two databases always agree.
7. The action is written to the log file, and the new version appears in the history list on the screen.

Restoring works the other way round: the program reads the list of files for the chosen version and copies each stored file back into an empty folder chosen by the user.

Relationships between the main classes

- One user has one profile and one set of settings.
- One user can own many projects and many repositories.
- One repository has many commits. If the repository is deleted, its commits and stored files are deleted with it (composition).
- Every commit has exactly one history record that describes its changes.
- A project can have many board cards, and a repository can have many releases, wiki pages and discussions.
- A release is always linked to the one commit it was made from, so its files never change.

3. Core features

3.1 Input/output

The program takes input from the user through forms on the screens: usernames, passwords, repository names, commit messages and so on. The user picks the folder to track with a folder chooser. Repospace then reads every file inside that folder, including files in sub-folders. For output, it writes restored versions into a new folder, creates ZIP

files for downloads, and shows results and messages on the screen. It can also serve a repository's pages as a small website on the user's own computer.

3.2 Processing logic

The main processing rule is hashing. Each file's content is passed through the SHA-256 algorithm, which produces a 64-character fingerprint. Two files with the same content always get the same fingerprint, and a changed file gets a different one. Repospace uses this to decide which files are new, which have changed and which were deleted, and to store each different file only once.

Other rules check who is allowed to do what. Only the owner can change whether a repository or project is public, and private items are never listed or downloadable for other users. These checks sit inside the model classes, not only in the screens, so another screen cannot skip them. Passwords are processed with PBKDF2, a slow hashing method with a random salt, so the real password is never stored.

3.3 Data structures

We used Java's generic collections:

- A **TreeMap** stores the list of files in a version (file path linked to its hash). It keeps the files in alphabetical order, so changes are always listed in the same order.
- An **ArrayList** holds results such as lists of commits, repositories and users read from the database.
- A **TreeSet** of dates stores the days on which a user made commits. Because it is sorted, it is easy to count streaks of days in a row.
- An **ArrayDeque** works as a stack for the Back button, remembering the screens the user came from.
- A **HashMap** remembers the names of repository owners on the Discover screen, so they are not looked up again.

3.4 Recursion and algorithms

Recursion is used to walk through a folder. The method looks at every item in a folder; when it finds a sub-folder, it calls itself again for that sub-folder, until every file at every

level has been found. The same idea is used to delete a whole folder tree, and the Markdown reader uses recursion to handle bold text inside italic text and similar cases.

The main algorithms are hashing, comparing two versions, sorting and searching. Comparing two versions takes each file in the new version and checks whether it exists in the old one and whether its hash is the same, and then checks the old version for files that are now missing. Lists are sorted (for example, files by path and activity by date), and users and public repositories can be searched by part of a name.

3.5 Logging and reporting

Repospace keeps three kinds of records. A log file records important actions, such as commits, restores, downloads and errors, with the date and time. Every sign-in attempt, successful or not, is saved in its own database. Each commit has a history record that reports exactly which files were added, modified or deleted. On the user's page, Repospace also reports statistics such as the number of repositories and commits, the longest streak of days with commits and the most used programming language.

3.6 Concurrency

Saving a commit can take time when a folder has many files, so it runs on a separate background thread. The window stays responsive, and a message appears when the commit is finished or if it fails. The small local website for repository pages runs on its own group of four threads, so several pages can be served at once.

4. Implementation

4.1 Code organisation

The code is divided into separate packages so that each part of the system has its own place.

Frontend (user interface). The screens are FXML layout files, designed separately from the Java code, with two style sheets for the light and dark themes. Each screen has its own controller class, which reacts to button clicks and passes the work to the model classes. There are twelve screens, from Login and Register to Repositories, Discover and Settings.

Backend (model and logic). The model package holds the classes that describe the real things in the system, such as User, Repository, Commit and History, and the rules about them. The util package holds helper classes that are shared by many parts of the program, such as the snapshot store, file helpers, password hashing, Markdown reading and logging.

Database (data storage). The DatabaseConnection class manages four SQLite database files: one for users and profiles, one for sign-in attempts, one for projects, repositories, commits and history, and one for the list of files in each commit. The file contents themselves are kept in a storage folder for each repository. Because SQLite stores everything in ordinary files, the data is still there after the program is closed, and no database server has to be installed.

4.2 Java best practices

- **Clear names.** Classes, methods and variables have descriptive names, such as Repository, pendingChanges and changeVisibility.
- **Comments.** Every class and public method has a short comment that explains what it does.
- **Encapsulation.** Fields are private and are only changed through methods, which check the input first.
- **Error handling.** Invalid input, such as a username with spaces or a password shorter than 8 characters, produces a clear message instead of a crash. Errors are caught and written to the log file.
- **Safe database access.** Every database query uses prepared statements, which protects against SQL injection.
- **Safe file handling.** Restoring and downloading refuse any file path that would write outside the chosen folder, and a ZIP file is only saved once it is complete.
- **Build tool.** Maven downloads JavaFX, the SQLite driver and JUnit automatically, so the project builds the same way on every computer.

4.3 GUI interface

The JavaFX interface was designed to be simple for beginners. It opens on the login screen, where the user signs in with a username or e-mail address and a password, or creates a new account. After signing in, a dark sidebar on the left gives access to Home, Projects, Repositories, Discover, My page, Edit profile, Settings and Log out.

The Repositories screen is the heart of the program. At the top, the user creates a new repository by giving it a name, choosing public or private and browsing to a folder. Below that, the user selects a repository, types a commit message and presses Commit, or presses Check changes to preview what has changed. The history list shows every version with its date and message, and buttons allow the user to view files, restore a version, clone the latest version, download it as a ZIP, change its visibility or delete it. Buttons stay greyed out until they can be used, which guides new users. Both a light and a dark theme are available in Settings.

5. Testing and evaluation

5.1 Sample files and test cases

We created a sandbox folder in the project with our own test material. The sample-project folder is a small, normal project with a README, a notes file, a Java source file and a settings file. It is used to make a first commit, edit a file, check the changes, commit again and restore the old version. The edge-cases folder holds awkward files that must survive a commit and restore without any change: an empty file, a binary file, a file with spaces in its name, a file five folders deep and an empty folder.

The system was tested in three ways:

- **Unit tests** checked the helper classes on their own, for example that the SHA-256 hash of a known text is correct, that the same password with a different salt gives a different hash, and that a folder walk finds files in sub-folders.
- **Integration tests** checked the model classes working with real databases. Each test uses a fresh, temporary database so that real user data is never touched. These tests covered registering and logging in, creating projects and repositories, committing, viewing history, restoring, sharing and the GitHub-style features.
- **Manual tests** of the screens followed our demo script with two accounts: one user shares work and a second user searches for it and downloads it.

- a wrong password, a duplicate username and a username with spaces;
- trying to download another user's private repository;
- a damaged snapshot that tries to write a file outside the restore folder;
- a search that contains the special characters % and _;

NEED SCREENSHOT

5.2 Evaluation

- **Correctness:** The main tasks (register, log in, commit, view history, restore, share and download) are checked by the automated tests, and restored files are compared byte for byte with the originals, including binary files. The manual tests confirmed that a second user only sees public projects and repositories.
- **Efficiency:** A file that does not change is stored only once, no matter how many versions are saved, so storage grows only with the changes. Files are hashed in small pieces, so even large files use little memory. One weakness is that every commit reads every file in the folder again, so committing a very large folder takes longer.
- **Ease of use:** Users work with buttons and lists instead of typed commands. Clear messages explain mistakes, buttons are disabled until they make sense, restoring only writes into an empty folder, and deleting a repository never touches the user's original folder, so beginners cannot lose work by accident.
- **Limitations:** Changes are shown per file, not line by line; there are no branches or merging; empty folders are not tracked; and the language setting is saved, but the screens are only in English.

6. Documentation and demo

When Repospace starts, the login screen appears. The user signs in with a username or e-mail address and a password, or clicks the link to create a new account.

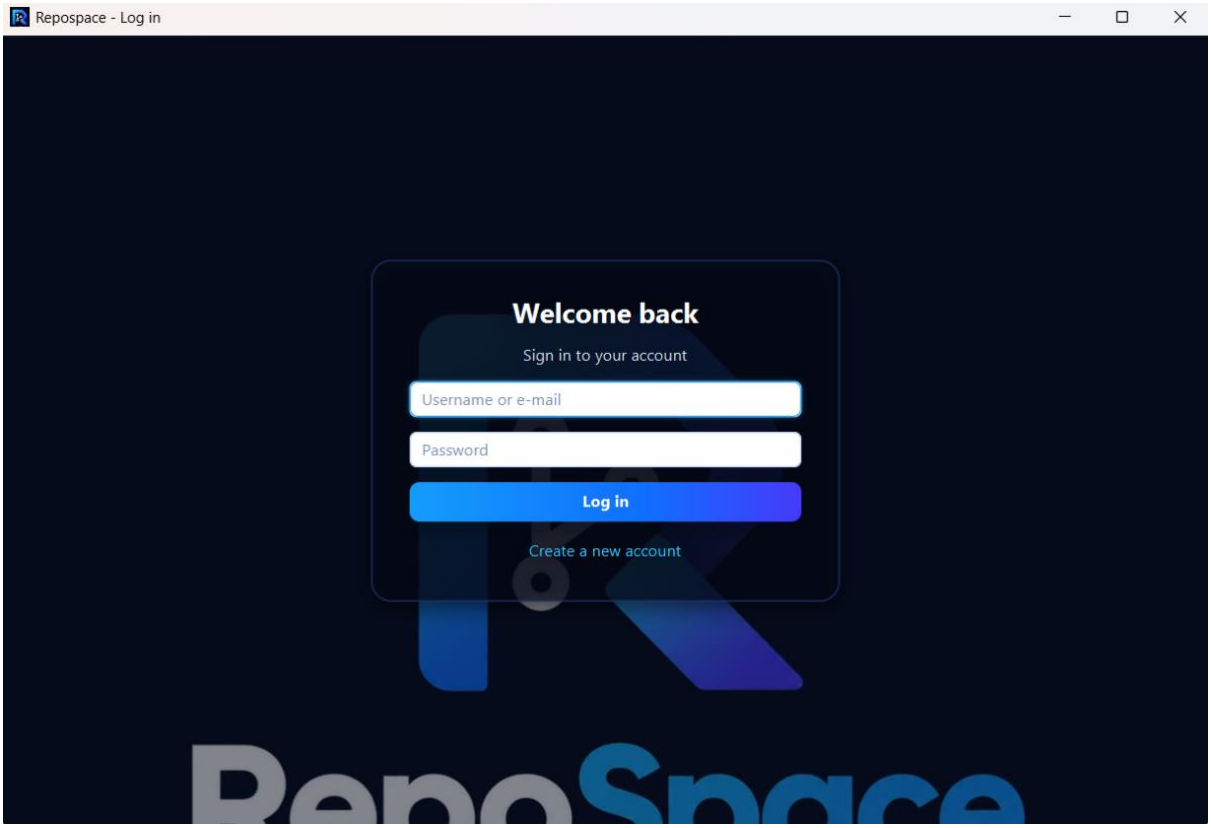


Figure 3: Log in screen.

On the Create an account screen, the hints in each box explain the rules: the username may not contain spaces, and the password must be at least 8 characters long and typed twice.

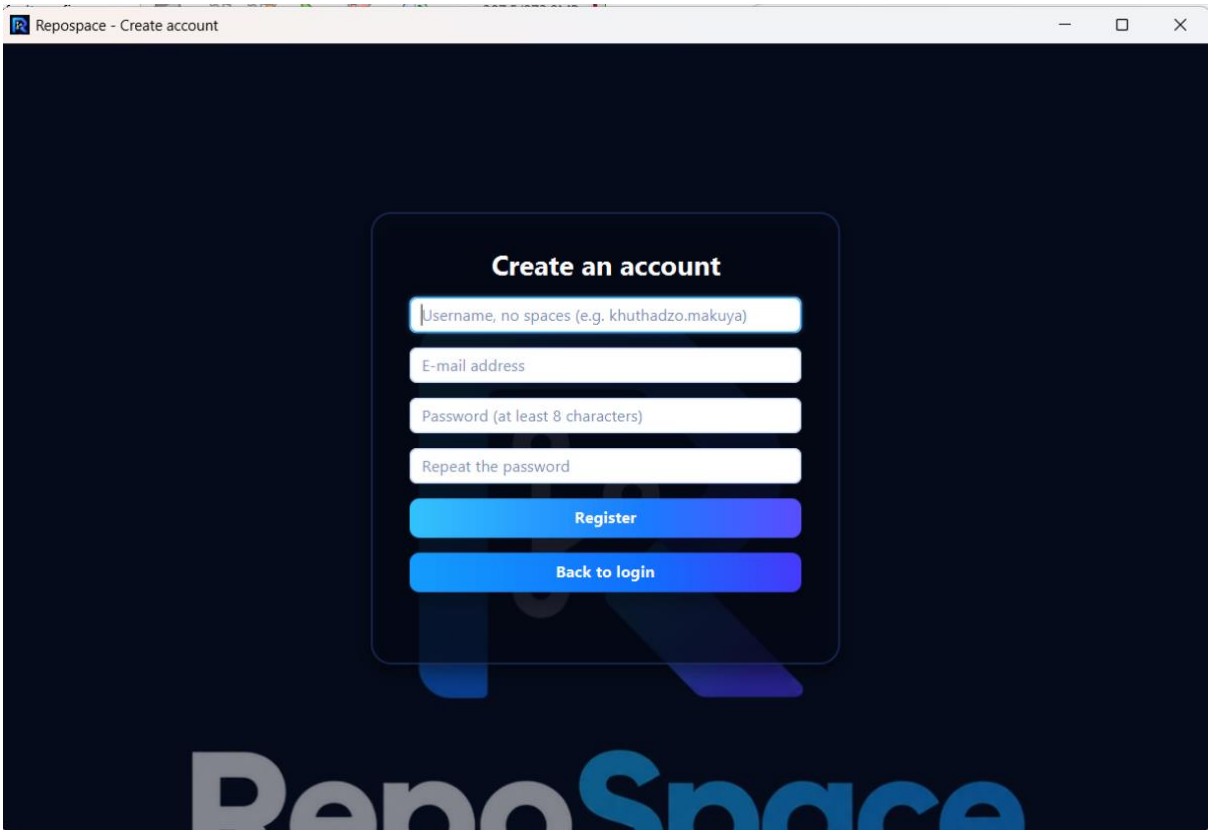


Figure 4: Create an account screen.

After registering, a message confirms that the account was created and that the user can now log in.

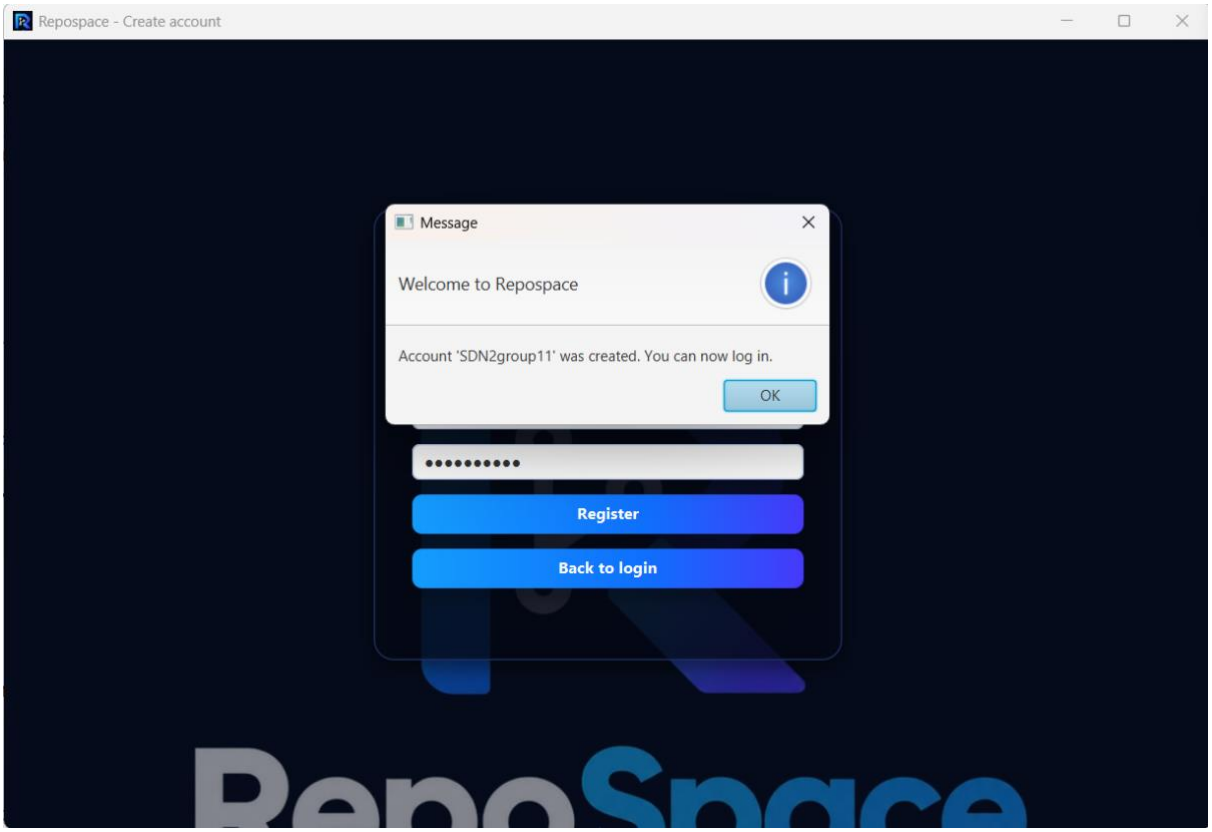


Figure 5: Confirmation that the account SDN2group11 was created.

After logging in, the Home screen shows how many projects and repositories the user has and how many commits they made in the last 7 days. The sidebar on the left leads to every part of the program.

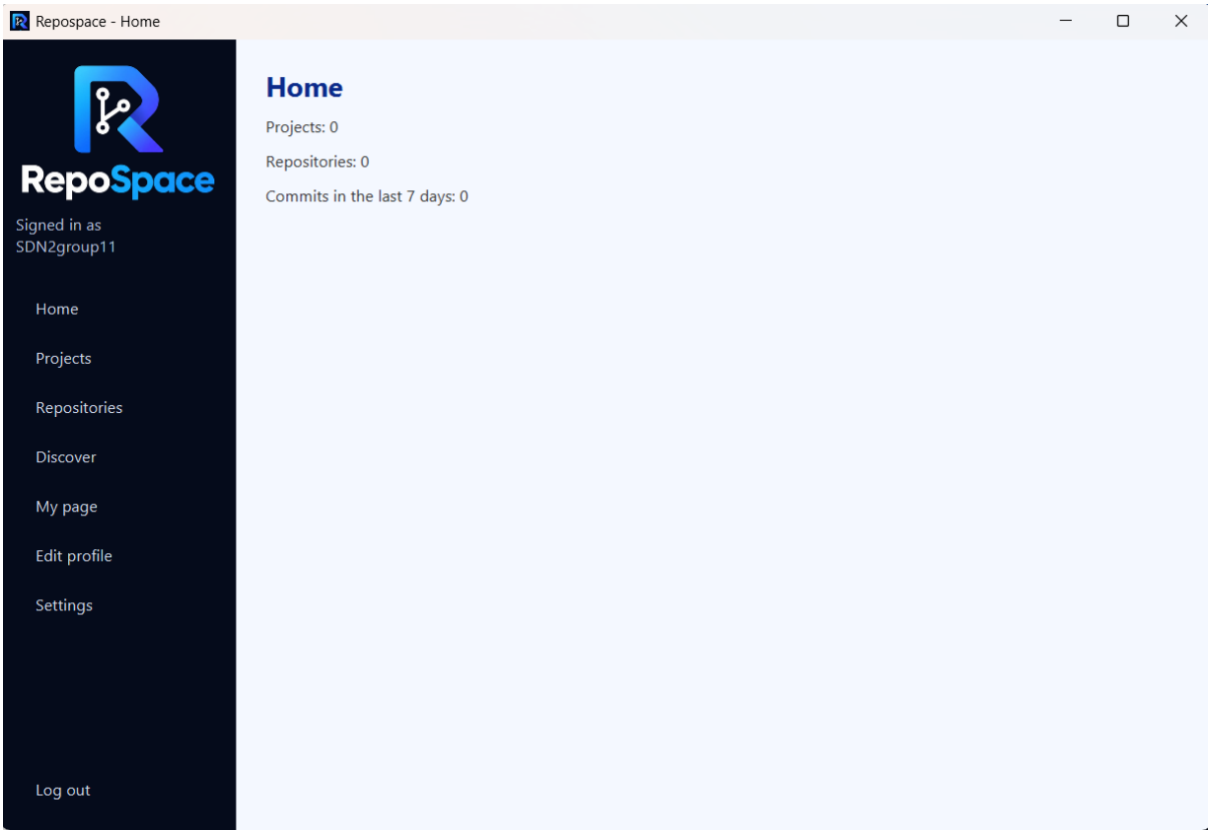


Figure 6: Home screen.

The Projects screen works like a to-do list for the user's work. Each project has a name, a description, a status and a visibility, and the Open board button shows a board with To do, In progress and Done columns.

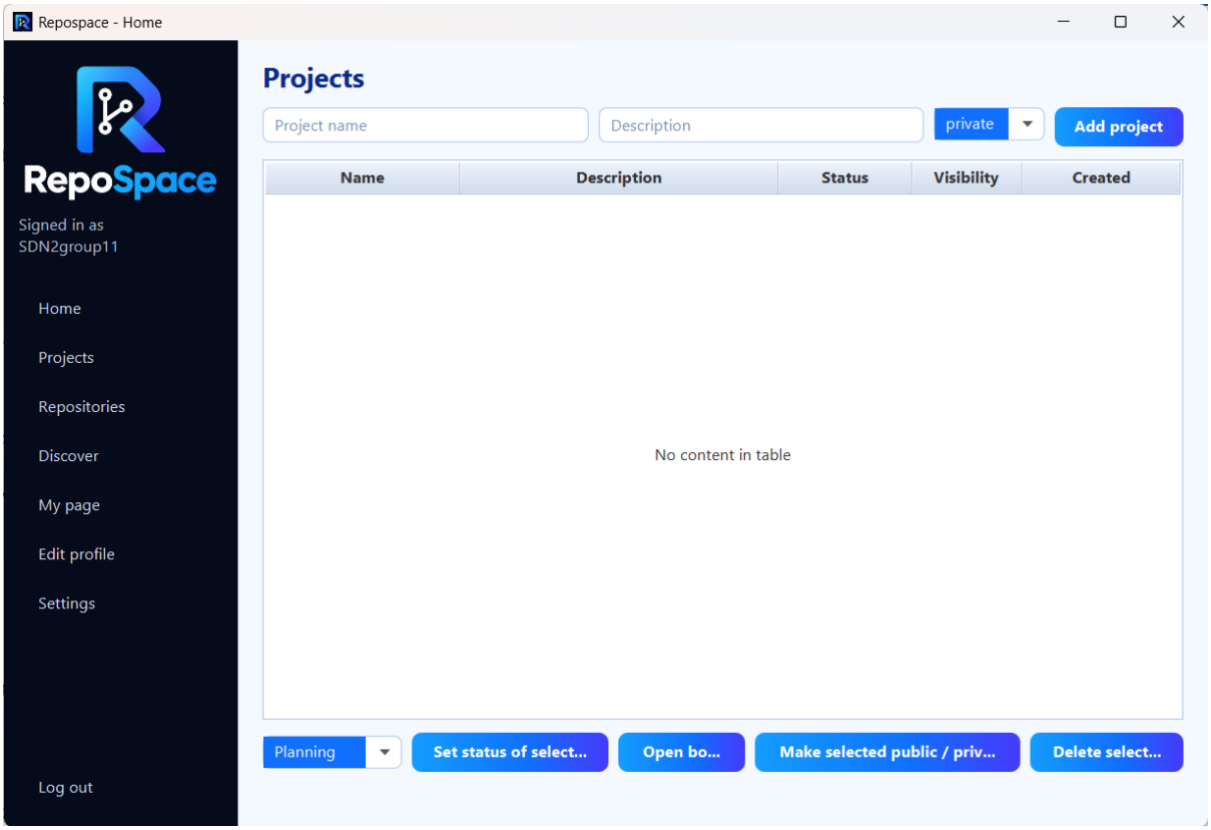


Figure 7: Projects screen.

The Repositories screen is where version control happens. The form at the top creates a new repository that tracks a folder, and the area below is used to commit, view the history and work with older versions.

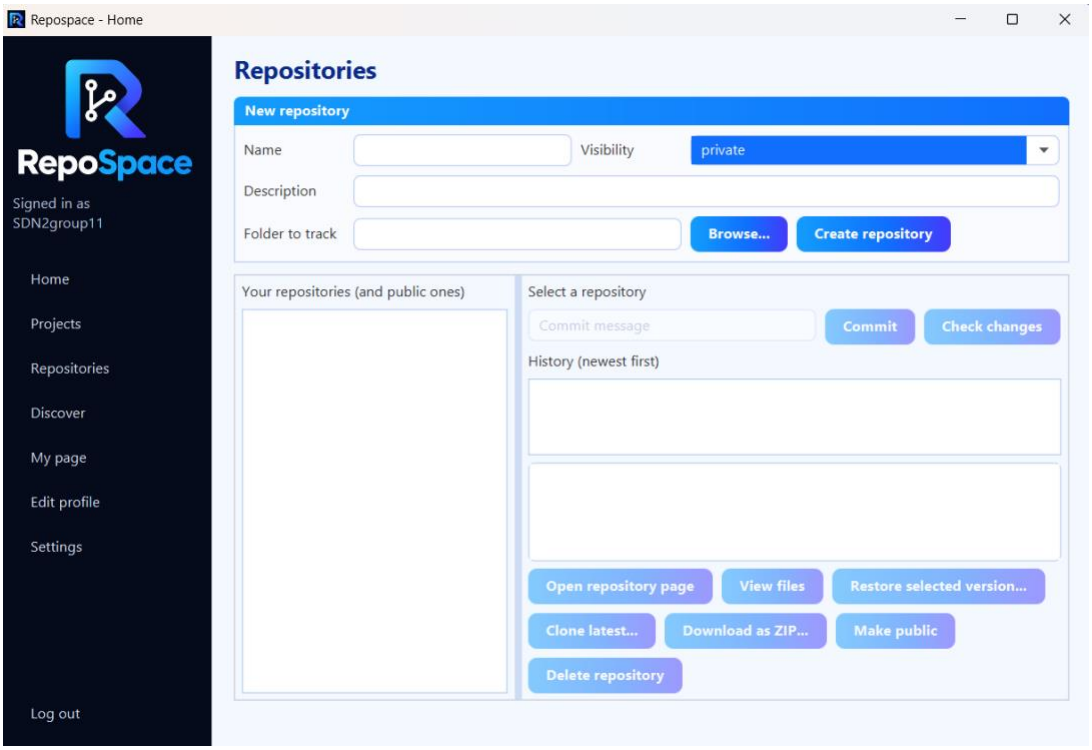


Figure 8: Repositories screen before a repository is selected.

After the first commit, the version appears in the history as v1 with its date and message, and the files in the latest version are listed below it.

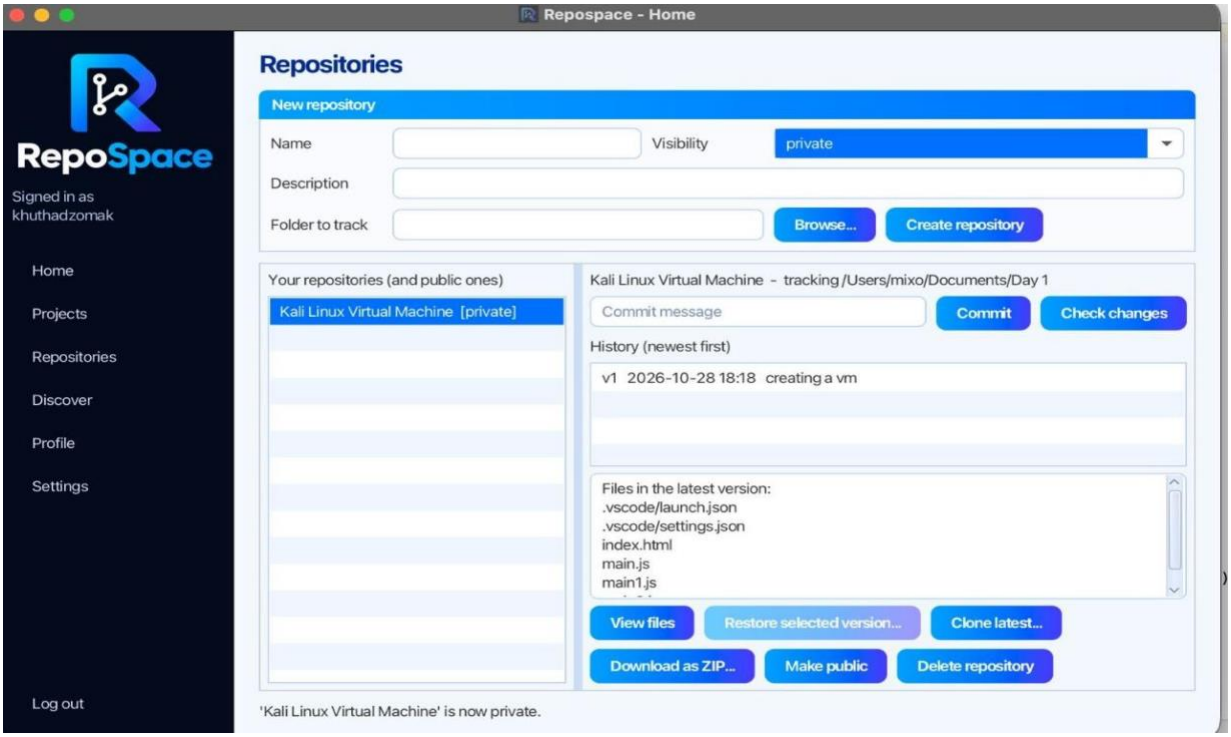


Figure 9: The first commit saved as v1.

The Discover screen lets users search for other people or for public repositories by name, description or topic.

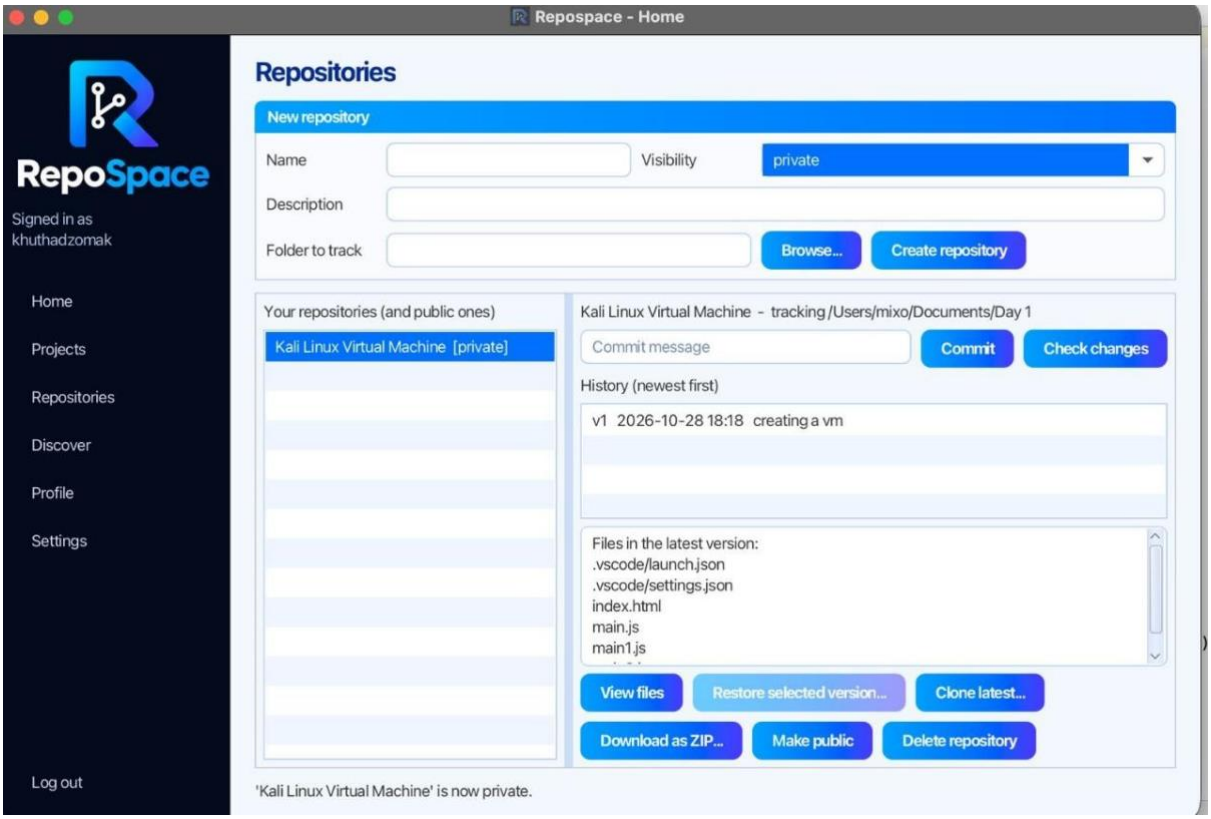


Figure 10: Discover screen.

When a second user searches for the first user, only the public project and public repositories are shown. The private project does not appear, and the public repositories can be downloaded as a ZIP or into a folder.

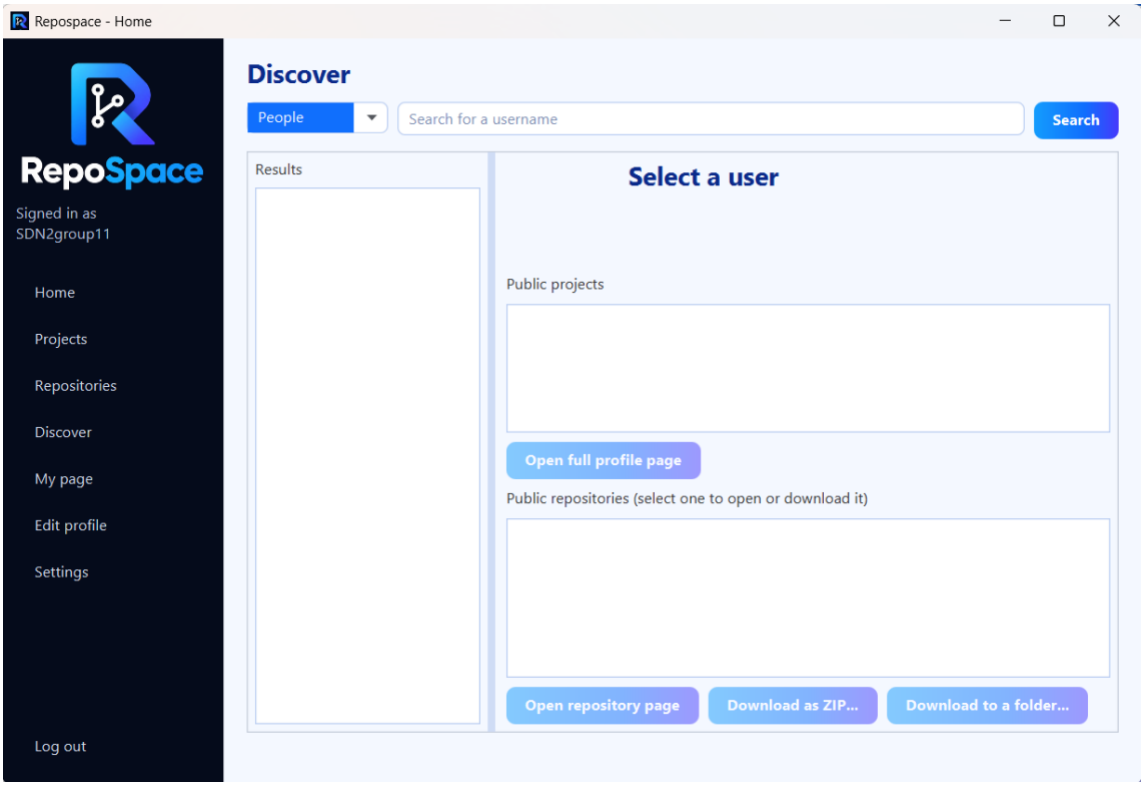


Figure 11: A second user sees only the public work of khuthadzomak.

My page shows how the user looks to others: statistics badges (repositories, commits, streak, top language, releases and sponsors), a profile README, top languages and pinned repositories.

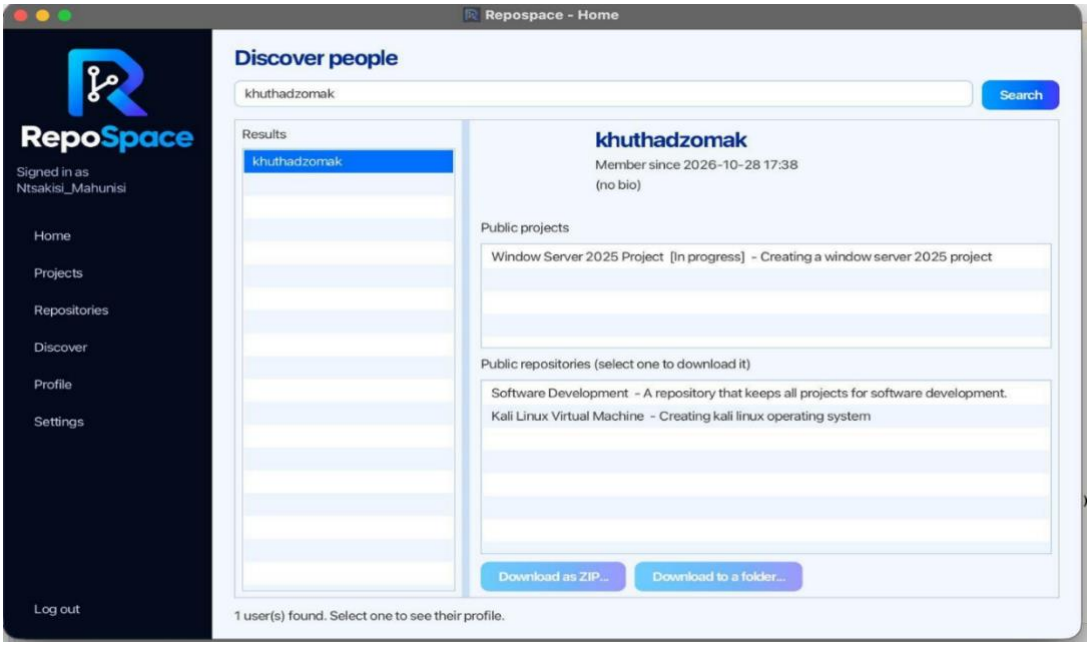


Figure 12: My page with statistics badges.

On the Edit profile screen, the user changes their bio, chooses who can see their profile, writes a short status and a profile README.

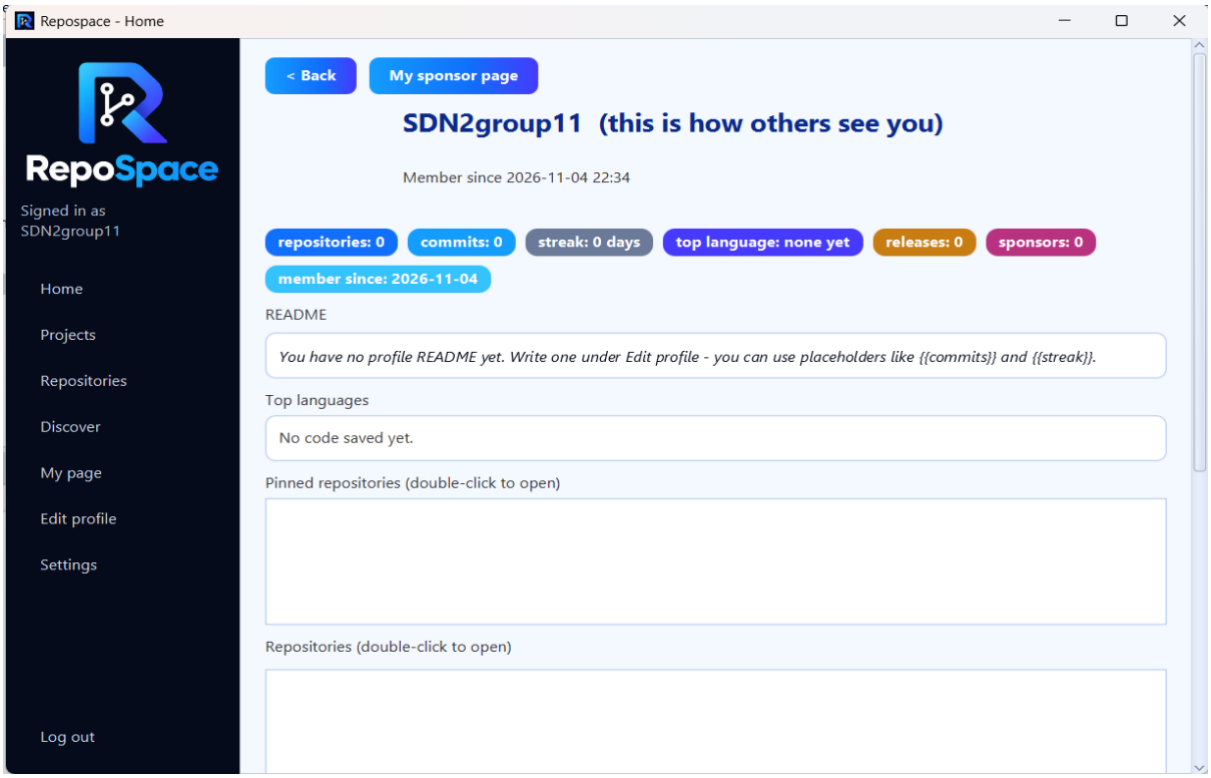


Figure 13: Edit profile screen.

The Settings screen lets the user switch between the light and dark themes, choose a language and turn activity notifications on or off.

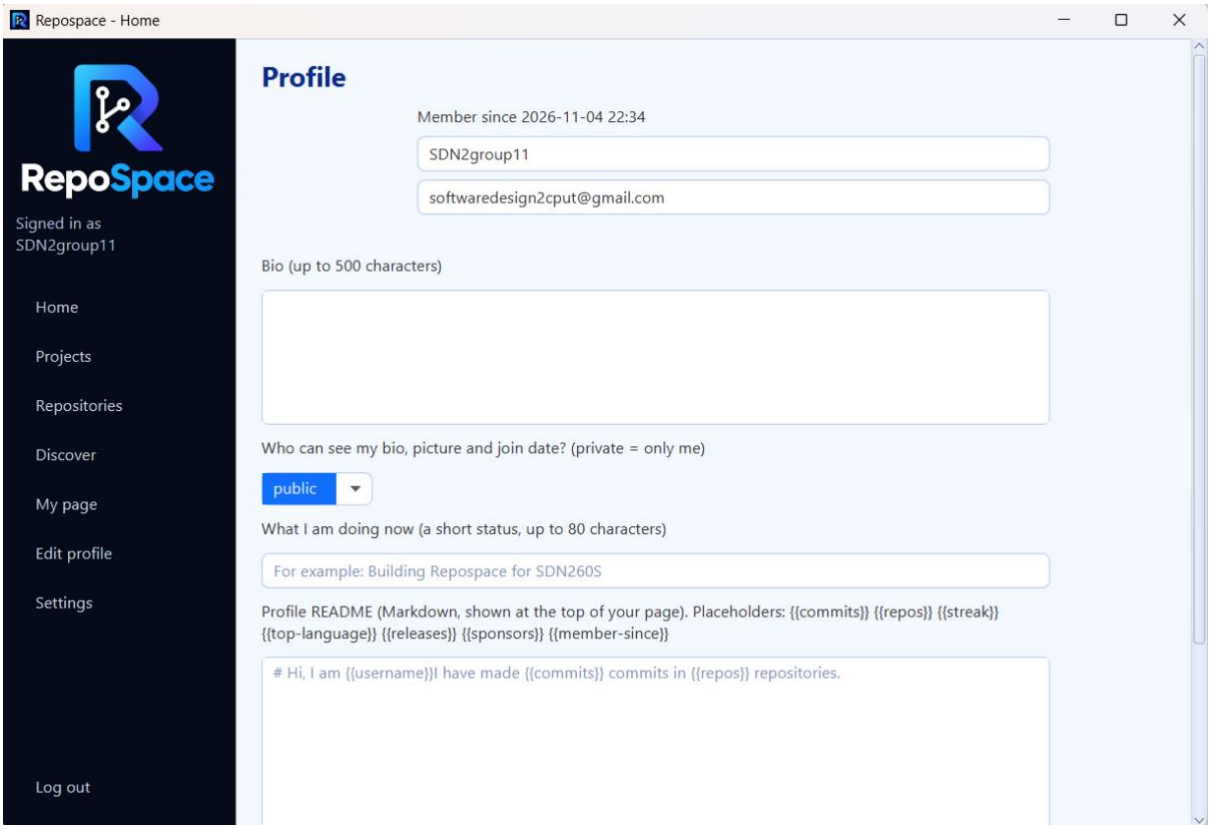


Figure 14: Settings screen.

7. Graduate attribute reflection

7.1 GA2 – Application of scientific and engineering knowledge

This project made us apply what we learned in Software Design to a real problem. We used object-oriented programming to split the system into classes with one job each, kept their data private, and used an interface so that different classes could be saved in the same way. We chose collections for how they behave: a sorted map for the list of files in a version, a sorted set for commit dates and a stack for the Back button. Recursion let us walk through folders of any depth. Hashing with SHA-256 gave every file a fingerprint, so changed files could be found without comparing them byte by byte, and salted password hashing kept accounts safe. We also used file handling, a relational database with transactions, a background thread for slow work, and automated testing to prove that the system works, including in unusual cases.

7.2 GA7 – Sustainability and impact of engineering activity

Repospace uses resources carefully. Because each different file is stored only once, saving many versions of a project takes far less disk space than keeping a full copy of the folder every time; less storage means less hardware and less energy. The program runs

on an ordinary computer without a database server or an internet connection after the first build, and its small local web server only runs while the program is open. It also saves people's time: restoring an old version is much quicker than redoing lost work. The project has a positive social impact because it makes version control easier for beginners, and it protects privacy, since new projects and repositories are private by default and e-mail addresses are never shown to other users. Finally, the clear structure, comments and tests make the software itself sustainable, because it can be maintained and extended without being rewritten.

9. Conclusion

Repospace meets the aim of the project. It lets users track a folder, save numbered versions, see exactly what changed, restore any earlier version and share their public work with others, all through a simple graphical interface and without installing a database server.

Building it showed us how the parts of a real system depend on each other: the screens collect what the user wants, the model classes apply the rules, and the databases and storage folder keep everything safe between sessions. The idea of storing each file once under its hash was the key design choice, because it keeps storage small and makes it easy to detect changes.

The project gave us practical experience with object-oriented design, recursion, collections, file handling, databases, threads and testing. In the future, Repospace could be improved with line-by-line differences, branches and merging, and support for collaborators.

10. References

1. Chacon, S. and Straub, B. (2014) *Pro Git*. 2nd edn. Apress. Available at: git-scm.com/book
2. Oracle (2021) *Java SE 17 API Specification*. Available at: docs.oracle.com
3. OpenJFX (2024) *JavaFX 21 documentation*. Available at: openjfx.io
4. SQLite (2024) *SQLite documentation*. Available at: sqlite.org
5. National Institute of Standards and Technology (2015) *FIPS 180-4: Secure Hash Standard*. Available at: csrc.nist.gov
6. Moriarty, K., Kaliski, B. and Rusch, A. (2017) *RFC 8018: PKCS #5: Password-Based Cryptography Specification Version 2.1*. IETF. rfc-editor.org
7. Deitel, P. and Deitel, H. (2018) *Java How to Program, Early Objects*. 11th edn. Pearson.
8. Fowler, M. (2004) *UML Distilled: A Brief Guide to the Standard Object Modeling Language*. 3rd edn. Addison-Wesley.
9. Sommerville, I. (2016) *Software Engineering*. 10th edn. Pearson.